

Ticketia Pro

Documentacion Tecnica

Trabajo Fin de Master

Plataforma de IA Empresarial para PYMEs

Flask + PostgreSQL/pgvector + OpenAI + Docker

Alejandro Bravot | 2025

Indice de Contenidos

1. Vision General del Proyecto
2. Stack Tecnológico
3. Arquitectura e Infraestructura (Docker)
4. Modelo de Datos (Base de Datos)
5. Agente Principal - AgentExecutor
6. Tool Calling & Herramientas del Agente
7. RAG - Retrieval Augmented Generation
8. Sistema de Metricas (LLM Tracker)
9. Historial de Conversacion
10. MCP - Model Context Protocol
11. Agentes Proactivos (Scheduler)
12. Consejo Pyme - Multi-Agent Debate
13. Frontend y Rutas Web
14. Flujos Criticos End-to-End
15. Escalabilidad y Decisiones de Diseno

1. Vision General del Proyecto

Ticketia Pro es una plataforma SaaS de inteligencia artificial diseñada para pequeñas y medianas empresas (PYMEs). Su objetivo es proporcionar un asistente conversacional multimodal capaz de gestionar citas, generar presupuestos, producir material de marketing, analizar tickets/facturas y ofrecer asesoramiento empresarial mediante un consejo virtual de tres agentes IA con personalidades diferenciadas.

La arquitectura sigue el patron RAG (Retrieval-Augmented Generation) con pgvector, orquestacion de herramientas mediante OpenAI Function Calling, y agentes proactivos autonomos que se ejecutan diariamente en background.

Capacidades Principales

- Chat conversacional por texto y voz (Whisper STT) con memoria de sesion
- Generacion de contenido de marketing

Modulos del Proyecto

Modulo	Ubicacion	Responsabilidad
AgentExecutor	modules/agents/manager.py	Orquestador principal: RAG + tools
CalendarTools	modules/agents/tools.py	Gestion de citas en BD
EmbeddingsService	modules/services/embeddings.py	RAG: ingesta, chunking, retrieval
LLMTracker	core/llm_tracker.py	Metricas de coste y latencia
CouncilManager	modules/council/orchestrator.py	Debate multi-agente (3 personas)
MCPClient	core/mcp_client.py	Cliente Model Context Protocol (SSE)
Scheduler	run_scheduler.py	Agentes proactivos (cron diario)
HistoryService	modules/agents/history.py	Ventana de contexto (50 msgsg)

2. Stack Tecnológico

Backend

Tecnología	Version	Uso
Python	3.11	Lenguaje principal
Flask	3.x	Framework web (blueprints, sesiones)
Flask-SQLAlchemy	3.x	ORM para PostgreSQL/SQLite
Gunicorn + gevent	25.x	WSGI server con workers async
APScheduler	3.x	Tareas programadas (cron diario)
Flask-Limiter	4.x	Rate limiting por IP
Flask-Mail	0.10	Envío de emails via SMTP
pywebpush	latest	Notificaciones Web Push (VAPID)

Inteligencia Artificial

Modelo / API	Proveedor	Uso en el proyecto
GPT-4o	OpenAI	Chat, vision, orquestacion de tools
text-embedding-3-small	OpenAI	Vectorizacion para RAG (1536 dims)
DALL-E 3	OpenAI	Generacion de imagenes de marketing
Whisper-1	OpenAI	Transcripcion de audio a texto (STT)
Gen3a Turbo	Runway ML	Generacion de video (2-stage pipeli
DuckDuckGo Search	DDGS	Busqueda web para agentes (sin API)

Infraestructura y Base de Datos

Componente	Tecnología	Rol
Base de datos	PostgreSQL 15 + pgvector	Datos relacionales + busqueda vecto
Contenedores	Docker Compose (4 servicios)	web, db, mcp-sse, scheduler
ORM	SQLAlchemy	Modelos Python -> tablas SQL
Busqueda vectorial	pgvector (cosine distance)	RAG similarity search
PDF	FPDF + reportlab	Generacion documentos PDF
Presentaciones	python-pptx	Generacion archivos .pptx

Frontend

- Jinja2 templates (renderizado server-side)

- Tailwind

3. Arquitectura e Infraestructura (Docker)

Docker Compose - 4 Servicios

El proyecto se despliega con Docker Compose. Cada servicio es independiente y se comunica por red interna Docker.

Servicio	Imagen / Comando	Puerto	Rol
ticketia_db	pgvector/pgvector:pg15	5432	PostgreSQL + extension pgvector
ticketia_mcp	python mcp_server_sse.py	8001	Servidor MCP SSE (tools compartidas)
ticketia_app	/app/entrypoint.sh	5000	Flask + Gunicorn (2 workers)
ticketia_scheduler	python run_scheduler.py	-	Agentes proactivos (cron)

Dependencias entre Servicios

```
db (PostgreSQL) -- healthcheck: pg_isready
|
+-- mcp (starts after db healthy)
|     MCP_SSE_URL=http://mcp:8001/sse
|
+-- web (starts after db healthy + mcp started)
|     DATABASE_URL=postgresql://postgres:password@db:5432/ticketia_db
|
+-- scheduler (starts after db healthy + web started)
|     Cron: ejecuta run_daily_tasks() cada dia a las 09:00
```

entrypoint.sh - Startup Seguro

El script de arranque ejecuta db.create_all() UNA sola vez antes de lanzar Gunicorn, evitando race conditions cuando multiples workers arrancan simultaneamente.

```
#!/bin/sh
python -c "from app import app, db; app.app_context().__enter__(); db.create_all()"
exec gunicorn -w 2 -b 0.0.0.0:5000 --timeout 300 app:app
```

Volumenes Persistentes

- pgdata - Base de datos PostgreSQL (nunca se pierde al reiniciar)

- uploads_

4. Modelo de Datos (Base de Datos)

BusinessProfile - Usuario/Empresa

Campo	Tipo	Descripcion
user_phone	String(20) PK	Identificador unico (telefono)
email	String(120) unique	Credencial de login
business_name	String(100)	Nombre comercial del negocio
password_hash	String(200)	Hash bcrypt de la contraseña
plan_tier	String(20)	BASIC PRO ENTERPRISE
features	JSON	{"tickets_allowed": true, "bot_enab
system_prompt	Text	Instrucciones del agente IA (wizard
static_knowledge	JSON	sector, servicios, horario, tono, p
active_agents	JSON	["grant_hunter", "networker", ...]
agent_config	JSON	{"post_sales": {"allow_refunds": tr
push_subscription	Text	JSON PushSubscription para Web Push

Appointment - Citas

Campo	Tipo	Descripcion
business_phone	String(20) FK	Propietario del calendario
date	Date	Fecha de la cita (YYYY-MM-DD)
time	String(10)	Hora en formato HH:MM
client_name	String(100)	Nombre cliente (opcional)
client_phone	String(20)	Telefono cliente (opcional)

Restriccion: no pueden existir dos citas con el mismo (business_phone, date, time). La verificacion se hace en CalendarTools.book_appointment() antes del INSERT.

KnowledgeChunk - Embeddings RAG

Campo	Tipo	Descripcion
user_phone	String(20)	Propietario del chunk
source_type	String(50)	"wizard" "document"
source_name	String(255)	Nombre campo wizard o nombre del fi
content	Text	Fragmento de texto (~1600 chars con
embedding	Vector(1536)	Embedding de text-embedding-3-small

LLMCall - Metricas de IA

Campo	Descripcion
model	gpt-4o gpt-4o-mini dall-e-3 w
stage	chat_main chat_tool_followup co
prompt_tokens / completion_tokens	Tokens de entrada y salida

latency_ms	Milisegundos de respuesta
cost_usd	Coste estimado calculado segun tabl
success	True si la llamada tuvo exito, Fals

Otros Modelos

Modelo	Proposito
Ticket	Facturas/gastos: imagen, concepto,
ChatMessage	Historial conversacion: role (user/
Grant	Ayudas y subvenciones: titulo, sect
SynergyMatch	Matches B2B entre empresas: user_a,
Notification	Notificaciones in-app: titulo, mens
ActivityLog	Registro de acciones de agentes: ag
GeneratedDocument	Docs generados (PDF, imagen, PPT, v
Incident	Incidencias de postventa: order_id,

5. Agente Principal - AgentExecutor

AgentExecutor es el cerebro de la plataforma. Recibe el mensaje del usuario (texto o transcripción de audio), enriquece el contexto con RAG, ejecuta el ciclo de tool calling con GPT-4o y devuelve la respuesta final. Se instancia en cada petición HTTP y no mantiene estado entre requests (stateless por diseño).

Pipeline execute() - 6 Pasos

```
def execute():
    # 1. Si hay imagen y agente admin_redactor activo:
    if media_url and "admin_redactor" in active_agents:
        return _handle_image_direct_processing()

    # 2. Guardar mensaje del usuario en BD
    HistoryService.save_interaction(phone, "user", user_message)

    # 3. Construir contexto: historial + RAG
    history = HistoryService.get_recent_history(phone, limit=10)
    rag_prompt = _build_rag_system_prompt(user_message)
    messages = [{"role": "system", "content": rag_prompt}] + history

    # 4. Primera llamada al LLM
    response = client.chat.completions.create(
        model="gpt-4o", messages=messages,
        tools=TOOLS_SCHEMA, tool_choice="auto"
    )
    track(phone, "gpt-4o", "chat_main", response, latency_ms)

    # 5. Si el LLM quiere ejecutar una herramienta:
    if response.choices[0].finish_reason == "tool_calls":
        return _process_tool_calls(messages, tool_calls)

    # 6. Guardar y devolver respuesta directa
    HistoryService.save_interaction(phone, "assistant", content)
    return content
```

Enriquecimiento RAG del System Prompt

Antes de cada llamada al LLM, el system prompt se enriquece dinámicamente con:

- Fecha y hora actual (para que el agente interprete fechas relativas correctamente)

- Top-5 ch

```
today = datetime.now().strftime("%A, %d de %B de %Y")
base = f"{system_prompt}\n\nFecha actual: {today}"

chunks = retrieve_chunks(phone, user_message, top_k=5)
if chunks:
    context = "\n".join(f"- {c}" for c in chunks)
```



```
return f"{base}\n\nCONTEXTO RELEVANTE:\n{context}"
```

Segunda Llamada al LLM (Tool Followup)

Cuando el agente ejecuta una tool, el resultado se agrega al historial de mensajes y se hace una segunda llamada al LLM SIN tools disponibles. Esto obliga al modelo a formular la respuesta final en lenguaje natural usando el resultado obtenido.

```
# Despues de ejecutar todas las tools:
messages.append({"role": "tool", "tool_call_id": tc.id, "content": result})

# Segunda llamada para sintetizar
final = client.chat.completions.create(model="gpt-4o", messages=messages)
track(phone, "gpt-4o", "chat_tool_followup", final, latency_ms)
return final.choices[0].message.content
```

6. Tool Calling y Herramientas del Agente

El patron de Tool Calling (Function Calling de OpenAI) permite que el LLM decida autonomamente cuando necesita ejecutar una accion del mundo real. El desarrollador define el contrato JSON (schema) de cada herramienta; el modelo rellena los argumentos y devuelve una llamada estructurada. Anadir una herramienta nueva no requiere modificar el orquestador: solo hay que registrar el schema en `TOOLS_SCHEMA` y el handler en `_process_tool_calls()`.

Herramientas Disponibles

Herramienta	Parametros Requeridos	Funcion Python
check_availability	date (YYYY-MM-DD)	CalendarTools.check_availability()
book_appointment	date, time	CalendarTools.book_appointment()
create_proposal_from_last_image	ninguno	_tool_create_proposal_from_last_ima
create_proposal_from_text	client_name, items, total	_tool_create_proposal_from_text()
generate_marketing_material	prompt, format	_tool_generate_marketing()
handle_customer_service	issue_summary	PostSalesAgent.handle_inquiry()

Ejemplo: Schema JSON de book_appointment

```
{
  "type": "function",
  "function": {
    "name": "book_appointment",
    "description": "Reserva una cita. Solo fecha y hora son obligatorios.",
    "parameters": {
      "type": "object",
      "properties": {
        "date": {"type": "string", "description": "YYYY-MM-DD"},
        "time": {"type": "string", "description": "HH:MM"},
        "client_name": {"type": "string", "description": "Opcional"},
        "phone": {"type": "string", "description": "Opcional"}
      },
      "required": ["date", "time"]
    }
  }
}
```

Flujo completo de Tool Execution

```
Usuario: "Reserva el viernes a las 10"

-> LLM#1 decide: finish_reason = "tool_calls"
tool_calls = [{
  "name": "book_appointment",
  "arguments": {"date": "2026-04-24", "time": "10:00"}
}]
```

```
-> Python ejecuta: CalendarTools.book_appointment(
    date="2026-04-24", time="10:00",
    business_phone=user_phone, client_name="", phone=""
)
-> result = "Cita confirmada para el 2026-04-24 a las 10:00."

-> messages.append({"role": "tool", "content": result, ...})

-> LLM#2 sintetiza:
    "Listo! He reservado el viernes 24 de abril a las 10:00. ¿Necesitas algo mas?"
```

Escalabilidad del sistema de tools

Añadir una nueva capacidad al agente requiere solo 3 pasos:

1. Definir el schema JSON en `TOOLS_SCHEMA` (contrato de parametros)
2. Implementar la funcion Python que ejecuta la logica
3. Agregar un elif en `_process_tool_calls()`

El LLM aprende a usar la herramienta solo por la descripcion del schema, sin reentrenamiento.

7. RAG - Retrieval Augmented Generation

El sistema RAG permite al agente consultar documentos propios del negocio en tiempo real. En lugar de reentrenar el modelo (costoso e inflexible), los documentos se fragmentan, se vectorizan con embeddings y se almacenan en pgvector. En cada consulta, los fragmentos mas similares semanticamente se inyectan en el system prompt del LLM.

Parametros de Chunking

Parametro	Valor	Justificacion
CHUNK_SIZE	1600 chars (~400 tokens)	Sweet spot para text-embedding-3-sm
CHUNK_OVERLAP	400 chars (~100 tokens, 25%)	Evita perder informacion en limites
Separadores	["\n\n", "\n", ". ", " ", ""]	Jerarquico: prioriza parrafos > lin
Min chunk size	50 chars	Descarta fragmentos demasiado corto
top_k retrieval	5 chunks	Balance entre contexto relevante y

Algoritmo de Chunking Recursivo

A diferencia del chunking fijo por caracteres, el chunking recursivo respeta las unidades semanticas naturales del texto:

```
def _split_text(text, chunk_size=1600, overlap=400):
    separators = ["\n\n", "\n", ". ", " ", ""]

    def _recursive(text, seps):
        sep = seps[0]
        if not sep: # Caso base: partir por chars con overlap
            while start < len(text):
                yield text[start : start+chunk_size]
                start += chunk_size - overlap
            return
        # Caso recursivo: partir por separador
        parts = text.split(sep)
        current = ""
        for part in parts:
            candidate = current + sep + part
            if len(candidate) <= chunk_size:
                current = candidate
            else:
                yield current
                if len(part) > chunk_size:
                    yield from _recursive(part, seps[1:]) # bajar nivel
                else:
                    current = part
        # Anadir overlap entre chunks consecutivos
        result = list(_recursive(text, separators))
        for i, chunk in enumerate(result):
            if i > 0: chunk = result[i-1][-overlap:] + " " + chunk
        return [c for c in result if len(c) > 50]
```

Pipeline completo de Ingesta (PDF/TXT)

```
def ingest_document(user_phone, file_path, filename):
    # 1. Extracción de texto
    if filename.endswith(".pdf"):
        text = pymupdf4llm.to_markdown(file_path) # preserva estructura
    else:
        text = open(file_path).read()

    # 2. Chunking recursivo
    chunks = _split_text(text) # lista de strings

    # 3. Embeddings + guardado
    for chunk in chunks:
        vector = embed_text(chunk) # list[float] * 1536
        db.session.add(KnowledgeChunk(
            user_phone=user_phone,
            source_type="document",
            source_name=filename,
            content=chunk,
            embedding=vector
        ))
    db.session.commit()
```

Retrieval por Similitud Coseno (pgvector)

```
def retrieve_chunks(user_phone, query, top_k=5):
    # Evitar embedding de query si no hay chunks
    if KnowledgeChunk.query.filter_by(user_phone=user_phone).count() == 0:
        return []

    query_vector = embed_text(query) # 1536 floats

    results = (
        KnowledgeChunk.query
        .filter_by(user_phone=user_phone)
        .order_by(KnowledgeChunk.embedding.cosine_distance(query_vector))
        .limit(top_k)
        .all()
    )
    return [r.content for r in results] # top 5 fragmentos
```

Distancia coseno en pgvector: valores entre 0 (idénticos) y 2 (opuestos). Un umbral típico de relevancia es < 0.5. pgvector usa el operador <=> para la distancia coseno, aprovechando índices HNSW o IVFFlat para búsquedas sublineales en colecciones grandes.

8. Sistema de Metricas (LLM Tracker)

El LLMTracker registra automáticamente cada llamada a modelos de IA con tokens consumidos, latencia, coste estimado y resultado. Los datos se persisten en la tabla LLMCall y se visualizan en la pagina /metrics con graficas de Chart.js.

Tabla de Precios (Abril 2025)

Modelo	Coste Input	Coste Output	Unidad
gpt-4o	\$2.50	\$10.00	por 1M tokens
gpt-4o-mini	\$0.15	\$0.60	por 1M tokens
dall-e-3	\$0.040	-	por imagen 1024x1024
whisper-1	\$0.006	-	por minuto de audio
gen3a_turbo	\$0.050	-	por segundo de video (~\$0.25/video

Context Manager `timed_track()`

El context manager mide latencia automáticamente y llama a `track()` en el bloque finally:

```
with timed_track(user_phone, "gpt-4o", "chat_main") as t:
    response = client.chat.completions.create(...)
    t["response"] = response # necesario para extraer tokens

# timed_track.finally() calcula latency_ms y llama a:
# track(user_phone, model, stage, response, latency_ms, success)
```

Metricas Disponibles en /metrics

- KPIs: coste total acumulado, tokens totales, latencia media, tasa de exito

- Grafica:

9. Historial de Conversacion

HistoryService gestiona la ventana de contexto del agente. Los mensajes se persisten en la tabla ChatMessage y se recuperan en orden cronologico para construir el contexto de cada llamada al LLM. Un mecanismo de auto-cleanup mantiene la ventana acotada.

Parametros

Constante	Valor	Proposito
MAX_HISTORY	50 mensajes	Threshold que dispara el cleanup au
KEEP_COUNT	40 mensajes	Mensajes que se conservan tras el c
limit (get_recent_history)	10 mensajes	Mensajes que se pasan al LLM en cad

Auto-cleanup

```
def save_interaction(phone, role, content, ...):
    db.session.add(ChatMessage(...))
    count = ChatMessage.query.filter_by(user_phone=phone).count() + 1
    if count > MAX_HISTORY: # > 50
        to_delete = count - KEEP_COUNT # cuantos borrar
        oldest = (ChatMessage.query
                  .filter_by(user_phone=phone)
                  .order_by(created_at.asc())
                  .limit(to_delete).all())
        for msg in oldest:
            db.session.delete(msg)
    db.session.commit()
```

CRITICO: Los mensajes de role="tool" DEBEN incluir los campos "name" y "tool_call_id" para cumplir el formato de la API de OpenAI. La ausencia de estos campos causa un error 400 en la segunda llamada al LLM. ChatMessage tiene columnas nullable para estos campos.

10. MCP - Model Context Protocol

MCP (Model Context Protocol) es un estandar abierto de Anthropic para conectar LLMs con herramientas externas de forma estandarizada. Ticketia lo usa para exponer tools de negocio (finanzas, agenda, email, estadísticas) que consumen el CouncilManager y otros agentes de forma desacoplada del agente principal.

Arquitectura MCP en el proyecto

Componente	Fichero	Rol
Servidor MCP SSE	mcp_server_sse.py	Expone tools via HTTP/SSE en puerto
Servidor MCP stdio	mcp_server.py	Fallback local: tools via stdin/std
Cliente MCP	core/mcp_client.py	Descubre y ejecuta tools del servid
Tools MCP	core/mcp_tools.py	Implementacion Python de las 6 tool

6 Tools expuestas por MCP

- get_financial_summary(user_phone) - resumen de tickets y gastos

- get_app

Transporte: SSE vs stdio

```
class TicketiaMCPClient:
    def execute_agent_loop(self, prompt, user_message):
        if self._sse_url: # Docker: MCP_SSE_URL definida
            return self._run_with_sse(prompt, user_message) # ~5ms latencia
        else:
            # Desarrollo local
            return self._run_with_stdio(prompt, user_message) # ~500ms (fork)
```

Ciclo de ejecucion MCP (_agent_loop)

```
async def _agent_loop(session, system_prompt, user_message):
    # 1. Descubrir tools disponibles en el servidor MCP
    tools_result = await session.list_tools()
    openai_tools = [convert_to_openai_schema(t) for t in tools_result.tools]

    # 2. Primera llamada LLM con tools MCP
    response = await client.chat.completions.create(
        model="gpt-4o",
        messages=messages,
        tools=openai_tools
    )

    # 3. Si hay tool calls, ejecutar via MCP
    for tc in tool_calls:
```

```
        result = await session.call_tool(tc.name, tc.arguments)
        messages.append({"role": "tool", "content": result.content, ...})

# 4. Segunda llamada para sintetizar resultados
return await client.chat.completions.create(model="gpt-4o", messages=messages)
```

11. Agentes Proactivos (Scheduler)

Los agentes proactivos son módulos autónomos que se ejecutan diariamente a las 09:00 AM vía APScheduler. Cada agente se activa individualmente por usuario desde el Marketplace. Todos generan notificaciones in-app y/o Web Push al terminar.

Agentes Disponibles

Agente (ID)	Clase	Frecuencia	Funcion Principal
grant_hunter	GrantHunterAgent	Diaria	Detecta subvenciones por sector y n
business_health	BusinessCoachAgent	Diaria	Analiza varianza de ingresos y gene
networker	SynergyAgent	Diaria	Detecta oportunidades B2B entre usu
post_sales_service	PostSalesAgent	Por evento	Gestiona devoluciones y quejas de c
marketing_generator	MarketingAgent	Por peticion	Genera imagenes, PPT y video en bac
admin_redactor	AdminAssistantAgent	Por imagen	Procesa imagenes -> presupuesto PDF

Grant Hunter - Subvenciones

```
def check_new_grants(user):
    for grant in Grant.query.all():
        if user.phone in grant.notified_phones:
            continue # ya notificado
        if grant.sector_focus == "General":
            match = True
        elif user_sector in grant.sector_focus:
            match = True
        else:
            # IA evalua match semantico
            match = _ai_match(user_sector, user_location, grant)
        if match:
            pitch = _generate_pitch(user, grant) # GPT: 50 palabras
            NotificationService.send_in_app(user.phone, titulo, pitch)
            NotificationService.send_push(user.phone, titulo, pitch)
            grant.notified_phones.append(user.phone) # no volver a notificar
    db.session.commit()
```

PostSales Agent - Clasificacion de Intenciones

El agente de postventa clasifica el mensaje del cliente en 5 categorías y aplica políticas configuradas por el dueño del negocio:

Intencion	Politica Aplicada	Resultado
RETOUR	allow_refunds=True	Generar etiqueta PDF devolucion
RETOUR	allow_refunds=False	Redirigir a proceso de aprobacion
RETOUR	forbidden_items match	Denegar con explicacion
EXCHANGE	exchange_url definida	Enviar URL de cambio + instruccione
COMPLAINT	siempre	Alerta urgente al dueño + respuesta

STATUS	siempre	Consultar estado pedido y responder
--------	---------	-------------------------------------

Synergy Agent - Networking B2B

El agente de networking analiza el perfil de gasto de cada usuario y lo compara con otros usuarios para detectar sinergias comerciales (cliente-proveedor o alianza estrategica). Cuando el score IA supera 80/100, crea un SynergyMatch y notifica a ambas partes.

12. Consejo Pyme - Multi-Agent Debate

El Consejo Pyme es un sistema de debate estructurado entre 3 agentes IA con personalidades diferenciadas. Dado un dilema empresarial, cada agente opina desde su perspectiva, debaten entre si y finalmente un "Secretario" sintetiza un Plan de Accion. El resultado se transmite al frontend en tiempo real via Server-Sent Events (SSE).

Las 3 Personas del Consejo

Persona	Rol	Estilo	Objetivo
El Socio (Tiger)	Growth & Ventas	Agresivo, impaciente	Maximizar ventas YA
El Gestor (Owl)	Legal & Fiscal	Conservador, tecnico	Evitar multas, viabilidad
El Coach (Rocket)	Productividad	Practico, empatico	Trabajar menos / mejor

3 Rondas del Debate + Sintesis

```
async def run_session(topic, user_context):

    # RONDA 1: opiniones independientes
    for persona in [socio, gestor, coach]:
        yield {"type": "typing", "agent": persona.id}
        opinion = await _get_agent_opinion(persona, topic)
        yield {"type": "message", "text": opinion, "round": 1}

    # RONDA 2: debate, replicas al ver las otras opiniones
    yield {"type": "divider", "text": "Debate"}
    for persona in [socio, gestor, coach]:
        rebuttal = await _get_agent_rebuttal(persona, topic, history)
        yield {"type": "message", "text": rebuttal, "round": 2}

    # RONDA 3: sintesis del Secretario
    yield {"type": "divider", "text": "Conclusion"}
    plan = await _generate_synthesis(topic, transcript)
    yield {"type": "plan", "text": plan} # Markdown
```

Prompt de cada Persona

```
prompt = f"""
Eres {persona.name} ({emoji}).
Rol: {persona.role}
Personalidad: {persona.style}
Objetivo: {persona.goal}

Contexto del negocio: {user_context}
Dilema: "{topic}"

Da tu opinion breve (max 30 palabras).
Sé directo y fiel a tu personalidad.
"""
```

El CouncilManager usa AsyncOpenAI (cliente async). Las 3 llamadas de cada ronda se hacen secuencialmente para mantener el drama del debate. Con `asyncio.gather()` se podrian paralelizar, pero perderia el efecto de "pensando en tiempo real".

13. Frontend y Rutas Web

Paginas Principales

Ruta	Template	Descripcion
/	landing.html	Landing page publica
/dashboard	dashboard.html	KPIs, tickets recientes, activity I
/wizard	wizard.html	Configuracion del agente (multi-ste
/marketplace	marketplace.html	Activar/desactivar agentes proactiv
/knowledge	knowledge.html	Subir documentos al RAG
/agenda	agenda.html	Calendario visual (FullCalendar 6)
/chatbot-cliente	chatbot_cliente.html	Vista simulada de cliente final
/metrics	metrics.html	Graficas de uso LLM (Chart.js)
/documents	documents.html	Archivos generados por IA
/council	council.html	Consejo Pyme (SSE streaming)
/profile	profile.html	Cambio de contrasena, perfil

Chat Widget Global (base.html)

El widget de chat esta embebido en base.html y se renderiza en TODAS las paginas cuando el usuario esta autenticado. Es un componente flotante (fixed bottom-right) que se muestra/oculta con toggleChat(). El envio usa fetch() a POST /api/chat.

Chatbot Cliente (/chatbot-cliente)

Pagina dedicada que simula la experiencia de un cliente final interactuando con el bot. Visualmente diferenciada (full-page, cabecera con nombre del negocio, indicador "En linea"). Internamente usa el mismo endpoint /api/chat sin cambios en el backend.

Agenda Visual (/agenda)

Usa FullCalendar 6 (via CDN, gratuito) con tres vistas: mes, semana y lista. Los eventos se cargan via GET /agenda/events (JSON). Al clicar una cita se abre un modal con nombre del cliente, telefono, fecha y hora.

```
# GET /agenda/events
appointments = Appointment.query.filter_by(business_phone=user_phone).all()
return jsonify([
    "id": a.id,
    "title": a.client_name or "Cita",
    "start": f"{a.date}T{a.time}:00",
    "extendedProps": {"client_phone": a.client_phone or ""}
} for a in appointments])
```

Server-Sent Events (Consejo Pyme)

El streaming SSE permite enviar eventos al frontend sin mantener una conexion WebSocket. Flask usa

`stream_with_context()` + `Response(generator, mimetype="text/event-stream")`. El frontend usa `EventSource` API para recibir y renderizar los eventos en tiempo real.

14. Flujos Criticos End-to-End

Flujo A: Chat por Voz -> Agendar Cita

```
1. Usuario habla: "Agenda una reunion el viernes a las 10"
2. POST /upload_web_audio
  -> Guardar .webm en /static/uploads/
  -> Whisper-1: transcribe -> "Agenda una reunion el viernes a las 10"
  -> run_agent(texto, phone, profile)

3. AgentExecutor.execute()
  -> RAG: retrieve_chunks (puede retornar [])
  -> LLM#1: gpt-4o, tool_calls=[
    {name: "book_appointment",
     arguments: {date: "2026-04-24", time: "10:00"}}
  ]
  -> CalendarTools.book_appointment(date, time, business_phone)
    -> Check conflictos
    -> INSERT Appointment
  -> LLM#2: sintetiza -> "Listo! Reunion agendada el viernes..."

4. Cita disponible en GET /agenda/events
5. FullCalendar la muestra en el calendario
```

Flujo B: Subida Documento -> RAG

```
1. POST /knowledge/upload (multipart, PDF)
2. Guardar en /static/uploads/knowledge/{phone}/
3. HTTP Response INMEDIATA: "Indexando..."

4. [Background Thread]:
  -> pymupdf4llm.to_markdown(pdf) -> texto markdown
  -> _split_text(texto) -> [chunk1, chunk2, ...] (1600 chars, 25% overlap)
  -> Para cada chunk:
    embed_text(chunk) -> vector[1536]
    INSERT KnowledgeChunk(user_phone, source_name=filename, embedding)
  -> commit()
  -> NotificationService.send_in_app("Documento indexado: N fragmentos")

5. Proximo chat del usuario:
  retrieve_chunks(phone, query) -> top-5 por cosine distance
  Chunks inyectados en system_prompt -> respuesta informada
```

Flujo C: Consejo Pyme (SSE)

```
1. POST /api/council/stream: {"topic": "Debo abrir sucursal?"}
2. CouncilManager().run_session(topic, user_context)
```

RONDA 1 (async, secuencial):

```
-> yield {type:typing, agent:socio}  
-> LLM: "Abre ya. El mercado esta maduro." -> yield {type:message}  
-> yield {type:typing, agent:gestor}  
-> LLM: "Verifica licencias primero." -> yield {type:message}  
-> yield {type:typing, agent:coach}  
-> LLM: "Focaliza en sistemas antes de expandir." -> yield {type:message}
```

RONDA 2 (replica leyendo las opiniones anteriores):

```
-> yield {type:divider}  
-> Cada agente ve las 3 opiniones y rebate
```

RONDA 3 (sintesis):

```
-> yield {type:divider}  
-> Secretario: Plan de Accion Markdown  
-> yield {type:plan, text:"# Plan\n1. ..."}  

```

3. Frontend EventSource renderiza cada evento en tiempo real

15. Escalabilidad y Decisiones de Diseño

Por que es escalable

- Tool Calling desacoplado: anadir una nueva capacidad al agente = definir schema JSON + 1 elif. El LLM aprende a usarla por la descripcion, sin reentrenamiento.

- RAG con
Agregar

Decisiones de Diseño Justificadas

Decision	Alternativa Descartada	Razon
Chunking recursivo 1600/400	Chunking fijo por chars	Respetar unidades semanticas; 400 to
PDF -> Markdown (pymupdf4llm)	pypdf texto plano	Preserva encabezados y listas; mejor
Overlap 25%	Overlap 0% o 50%	20-25% es el rango optimo documenta
top_k=5 chunks	top_k=1 o top_k=20	5 fragmentos: contexto suficiente s
pgvector cosine	BM25, FAISS, ChromaDB	Integrado en PostgreSQL (sin infra
Gunicorn sync workers	FastAPI async	Flask sincrono es mas simple; async
Background threads	Celery + Redis	Sin infra adicional para el TFM; en
SSE vs WebSocket	WebSocket	SSE es unidireccional (servidor ->

Limitaciones Identificadas (Trabajo Futuro)

- Concurrencia: Gunicorn sync con 2 workers no escala a miles de usuarios concurrentes. Solucion: migrar llamadas LLM a cola Celery + Redis.

- Memoria
sesiones

Resumen Tecnico Ejecutivo

Aspecto	Tecnologia / Valor
Lenguaje	Python 3.11
Framework web	Flask 3.x + Gunicorn + gevent
Base de datos	PostgreSQL 15 + pgvector (Vector(15

LLM principal	GPT-4o (chat, vision, tool calling,
Modelos secundarios	DALL-E 3, Whisper-1, Runway Gen-3,
Orquestacion herramientas	OpenAI Function Calling + MCP (SSE/
Busqueda vectorial	pgvector cosine distance (top_k=5,
Concurrencia	Threading (background tasks), Async
Rate limiting	flask-limiter (30 req/min chat, 20
Notificaciones	In-app (DB), Web Push (PWA/VAPID),
Scheduler	APScheduler (cron 09:00 diario, con
Infraestructura	Docker Compose: 4 servicios (web, d
Frontend	Jinja2 + Tailwind CSS + JS vanilla